

La pila de NodeFlow: qué nos estamos salteando

Tauri 2 · React Flow · Rust · grafo propio en JSON. Qué capacidades existen, cuáles ya vienen en el core sin plugin, cuáles son realmente oficiales — y en qué orden conviene tocar algo.

Cada fila de este informe se sondeó contra la fuente autoritativa (npm, crates.io, GitHub) el 19/09/2026. Lo que la investigación delegada afirmó y no se sostiene está en la §6, con la prueba.

0 · Cómo leer esto

Marcas, siempre visibles: verificado lo comprobé contra el registro o el código · a probar existe y es plausible, pero no lo corrí en tu máquina · descartado no existe, está muerto o no sirve para un producto distribuible.

descartado Correcciones a lo que devolvió la investigación delegada	
Afirmación recibida	Realidad medida
<code>tauri-plugin-frameless-window</code> , «plugin oficial» con efectos Mica/Acrylic	No es oficial: es de un particular (<code>github.com/insd47/...</code>) y tiene 195 descargas en total. No hace falta: los efectos vienen en el core (§3).
<code>tauri-plugin-wdio</code> y <code>tauri-plugin-wdio-webdriver</code> , «plugins oficiales»	Existen y son serios (1.4.0, 87k y 128k descargas), pero son de WebdriverIO (<code>github.com/webdriverio/desktop-mobile</code>), no de Tauri. El oficial para E2E es <code>tauri-driver</code> (<code>tauri-apps/tauri</code>).
<code>sled</code> «1.0.0-alpha.124»	La última publicada es 0.34.7 , del 11/10/2024 : 23 meses sin releases. La fecha coincide, la versión no.
<code>dashmap</code> «7.0.0-rc2» y <code>tauri-specta</code> «2.0.0-rc.25» citados como versiones actuales	Son release candidates : la estable de dashmap es 6.2.1 y la de tauri-specta 1.0.2 . Un RC no es una versión para meter en producción sin decidirlo.
URL de <code>cmdk</code> → <code>codingdunia.com</code> ; <code>zustand</code> → <code>zustand.site</code>	Paquetes reales, URLs equivocadas . Las oficiales: <code>github.com/pacocoursey/cmdk</code> y <code>github.com/pmndrs/zustand</code> .
<code>rustfmt</code> y <code>clippy</code> citados como crates de crates.io (2017/2018)	El consejo es correcto (<code>cargo fmt --check</code> y <code>cargo clippy -D warnings</code> en CI) pero no son dependencias : vienen con el toolchain de Rust.
Evento de archivos soltados <code>tauri://file-drop</code>	En Tauri 2 el evento es <code>tauri://drag-drop</code> (la API JS es <code>onDragDropEvent</code>).

1 · Qué usa NodeFlow hoy (medido, no recordado)

Pieza	Versión en el proyecto	Última disponible	Estado
React Flow (reactflow)	11.11.4	@xyflow/react 12.11.6	rama congelada desde jun-2024
Tauri (Rust)	2.11.3	2.11.5 (15/09/2026)	2 parches atrás
Plugin oficiales en uso	log , single-instance , updater 2.11.0, process 2.3.1	—	de 25 oficiales, usás 4
ACL / capacidades	core:default , updater:default , process:default + 2 permisos de ventana	—	mínima y prolija
Política de seguridad de contenido (CSP)	"csp": null	—	hueco real
Frontend	React 19.0.1, Vite 6.2.3, Tailwind 4, Motion 12, motion , lucide-react	—	al día
Tests del frontend	ninguno (sólo tsc --noEmit)	—	hueco real
Backend	~11.000 líneas Rust, 30 módulos, 281 tests	—	con tests y CI
Grafo	propio: grafo.rs (1.338 líneas) + serde_json	—	sin dependencia externa

Ya tenés drag&drop habilitado en la ventana (dragDropEnabled: true) y el servidor MCP viaja empaquetado como recurso (bundle.resources).

2 · El hallazgo que más pesa: tu librería central está en la rama muerta

	versión	última publicación	línea
reactflow (lo que usás)	11.11.4	20/06/2024	congelada
@xyflow/react (su sucesora)	12.11.6	01/09/2026	85 releases desde 12.0.0 (jul-2024)

Dos años de arreglos, rendimiento y API que no estás recibiendo. **Ambas aceptan React ≥ 17**, así que la migración no es urgente por incompatibilidad: es una mejora disponible. La v12 además deja de traer subpaquetes sueltos (@reactflow/minimap , -node-resizer , -node-toolbar) y pasa a depender de @xyflow/system + su propio store con zustand .

Qué te llevás migrando

- **Rendimiento en lienzos grandes:** el render sólo de lo visible ya existe en v11 (onlyRenderVisibleElements) pero la v12 suma memorización de nodos y menos re-renders del store. Es justo lo que vas a necesitar cuando el lienzo pase de 200 nodos.

- **Piezas de UI que hoy escribís a mano:** `NodeToolbar` y `NodeResizer` ya existen como componentes (y en v11 también, pero como paquetes separados).
- **Sub-flows:** nodos que contienen un lienzo. Tu `ZonaNode` es exactamente eso, hecho a mano.
- **SSR y tipos más finos,** con utilidades nuevas (`useNodesData` , flujos computados).

Riesgo honesto de migrar

Es un renombre de paquete + cambios de API en tus nodos y en los tipos: **el trabajo no está en el `package.json` , está en `App.tsx` (4.812 líneas) y en 34 componentes.** Con la rama de trabajo y los árbitros que ya tenés, se hace en un worktree y no toca el grafo ni el backend. Si algo sale mal, volver atrás es revertir la rama.

3 · Tauri: lo que ya tenés pagado y no estás usando

Verifiqué las dependencias directas de `tauri` 2.11.5: trae `tray-icon` , `muda` (menús) y `window-vibrancy` (efectos Mica/Acrylic en Windows) como parte del core. O sea: **no hace falta ningún plugin de terceros** para bandeja, menús ni efectos de ventana.

Capacidad	Cómo se activa	Para qué en NodeFlow	Esfuerzo
Bandeja del sistema core	feature <code>tray-icon</code> + <code>TrayIconBuilder</code>	La app queda viva sin ventana; menú con «capturar idea», «abrir bóveda», «salir»	2 h
Menús nativos core	<code>muda</code> vía <code>tauri::menu</code>	Menú de app correcto (y atajos visibles), en lugar de sólo atajos propios	2 h
Efectos de ventana core	<code>window-vibrancy</code> : Mica/Acrylic	Aspecto nativo de Windows 11 sin tocar la UI	2 h
Archivos soltados core	evento <code>tauri://drag-drop</code> (ya tenés <code>dragDropEnabled</code>)	Soltar un PDF/imagen sobre el lienzo y que nazca un nodo	1 h
Canales IPC core	<code>tauri::ipc::Channel</code>	Streaming de tokens al nodo activo (hoy va por SSE propio)	a probar
Sidecar	<code>tauri-plugin-shell</code> (<code>Command.sidecar</code>)	Empaquetar el servidor MCP de Python y el TTS sin exigir Python instalado	3 h
CSP y endurecimiento	<code>app.security.csp</code> en <code>tauri.conf.json</code>	Cerrar el <code>csp: null</code> antes de que agentes externos toquen la app	2 h

Plugins oficiales que valen la pena (todos verificados, publicados el 13/09/2026)

Plugin	Versión	Para qué acá	Esfuerzo
<code>tauri-plugin-global-shortcut</code>	2.3.2	Atajo global para capturar una idea sin abrir la ventana: es <i>tu</i> caso de uso	3 h
<code>tauri-plugin-notification</code>	2.4.0	Avisar cuando el cerebro terminó una tarea o una investigación	3 h
<code>tauri-plugin-window-state</code>	2.4.1	Recordar tamaño, posición y maximizado entre sesiones	2 h
<code>tauri-plugin-autostart</code>	2.5.1	Arrancar con Windows (opcional, y con la bandeja tiene sentido)	2 h
<code>tauri-plugin-clipboard-manager</code>	2.3.3	Copiar/pegar nodos, exportar a markdown al portapapeles	2 h
<code>tauri-plugin-deep-link</code>	2.4.10	<code>nodeflow://</code> : abrir la app desde un link — la pieza que hace falta para el imán de leads	4 h
<code>tauri-plugin-store / -sql</code>	2.4.5 / 2.4.1	Persistencia tipada del lado oficial, si algún día querés dejar de escribir tus propios JSON	a probar
<code>tauri-plugin-stronghold</code>	2.3.2	Caja fuerte cifrada; hoy usás el llavero de Windows (decisión válida, esto es la alternativa)	a probar
<code>tauri-plugin-positioner</code>	2.3.4	Abrir la ventana pegada al cursor o a la bandeja	1 h

Distribución en Windows (y qué exige cada canal)

Canal	Qué exige	Veredicto
winget (<code>winget-pkgs</code>)	Manifiesto YAML + URL del instalador + hash. La firma ayuda pero no bloquea	el más barato y el de más alcance : <code>winget install NodeFlow</code>
Microsoft Store (MSIX)	Cuenta de desarrollador paga + empaquetado MSIX + firma	después, si hay usuarios
Chocolatey	Paquete <code>.nupkg</code> + script + checksum	sirve si el público es técnico
Scoop	Manifiesto JSON + SHA256	2 horas y quedás instalable en el mundo dev
Firma Authenticode	Certificado o Azure Trusted Signing (ya tenés Azure con crédito)	Prerequisito de la experiencia «programa serio»: sin eso, SmartScreen avisa

4 · El lienzo: lo que un editor de nodos suele tener y acá no

Pieza	Paquete / API	Para qué	Nota de verificación
Paleta de comandos (⌘K)	<code>cmdk</code> 1.1.1 + <code>fuse.js</code> 7.5.0	Toda acción de la app a dos teclas, con búsqueda difusa: es la mejora de UX más grande por hora invertida	verificado (MIT)
Avisos transitorios	<code>sonner</code> 2.0.8 (o <code>react-hot-toast</code>)	Confirmar «guardado», «propuesta aplicada», errores de motor	verificado
Estado del lienzo	<code>zustand</code> 5.0.15 (+ <code>immer</code> 11.1.18)	La v12 de React Flow ya usa <code>zustand</code> puertas adentro: alinearse evita dos modelos de estado	verificado (MIT)
Layout automático	<code>@dagrejs/dagre</code> 3.1.1	Reordenar el árbol por niveles cuando el lienzo se desordena (alternativa a tu <code>layout.ts</code>)	verificado MIT
Layout jerárquico avanzado	<code>elkjs</code> 0.12.0	Layout tipo Sugiyama, más ordenado en grafos con cruces	licencia EPL-2.0 OR GPL : cuidado en un producto que distribuís
Mini-mapa	<code><MiniMap /></code>	Orientación con lienzo grandes; son 5 líneas	existe en v11 y v12
Tests del frontend	<code>vitest</code> 5.0.1 + <code>@testing-library/react</code> 16.3.3	Hoy no hay ni un test de UI: cualquier refactor de <code>App.tsx</code> es a ciegas	verificado MIT
E2E de escritorio	<code>tauri-driver</code> 2.0.6 (oficial) o Playwright 1.63.0 sobre el dev server	Probar el flujo completo (dictar, crear nodos, guardar) sin manos	verificado
Colaboración / multi-dispositivo (tu fase móvil)	<code>yjs</code> 13.6.32 + <code>y-websocket</code> 3.1.0	CRDT: dos clientes editan el mismo grafo sin servidor central ni conflictos	verificado MIT
<code>automerge</code>	2.0.0-alpha.3	Alternativa CRDT en Rust/WASM	en alpha : no para producción

5 · Rust: fiabilidad, observabilidad y velocidad

5.1 · Si algún día hace falta una base (la que creíamos Kùzu)

Opción	Versión	Última publicación	Lectura
redb	4.3.0	15/09/2026	KV puro Rust, vivísima. Es la candidata natural si el JSON deja de alcanzar
rusqlite	0.40.2	08/08/2026	SQLite: SQL y transacciones, con el costo de un C embebido
fjall	3.1.10	30/08/2026	B-tree alternativo, activo
sqlx	0.9.0	21/05/2026	Sobredimensionado para hoy (async + macros)
sled	0.34.7	11/10/2024	descartado : 23 meses sin releases

Regla de la casa que conviene sostener: el JSON legible es lo que hace auditable tu estado (y lo que te deja inspeccionar la bóveda con Obsidian). Una base entra sólo si una medición muestra que el JSON ya no alcanza.

5.2 · Lo que sí mejoraría el día a día

Crate	Versión	Qué resuelve en tu proyecto	Esfuerzo
thiserror	2.0.20	Hoy todos los errores viajan como String : no se pueden distinguir ni testear por tipo	bajo
tracing + tracing-subscriber	0.1.44 / 0.3.23	Trazas estructuradas y filtrables por módulo: hoy usás log y depurás leyendo líneas	bajo
schemars + tauri-specta	1.2.2 / 1.0.2 (estable)	Generar los tipos TypeScript desde el Rust: contratos que no se pueden desincronizar. hay 2.0.0-rc: verificar cuál corresponde a Tauri 2	medio
proptest	1.11.0	Invariantes del grafo generadas al azar: tu ADR 0003 hecho test	medio
insta	1.48.0	Snapshots del JSON del lienzo: detecta cambios no intencionados en el formato	bajo
criterion	0.8.2	Medir carga/guardado del grafo antes de optimizar a ojo	bajo
cargo-nextest	0.9.145	Correr los 281 tests más rápido y en paralelo	bajo
cargo-mutants	27.1.0	Medir si tus tests detectan bugs de verdad (no cuántos son)	medio
cargo-deny / cargo-audit	0.20.2 / 0.22.2	Licencias y CVE en CI: con un producto que distribuís, esto deja de ser opcional	bajo
sccache	0.18.0	Builds de Windows y de CI mucho más rápidos	bajo
parking_lot / arc-swap / once_cell	0.12.5 / 1.9.2 / 1.21.4	Locks más rápidos y configuración que se relee sin bloquear	bajo
bincode / rmp-serde	3.0.0 / 1.3.1	Sólo para cachés internas: el estado del usuario queda en JSON legible	bajo

6 · Qué haría yo, en este orden

Movimiento 1 — Cerrar los dos huecos que ya son deuda (una tarde)

1. **CSP**: poner una política en `tauri.conf.json` (`csp: null` hoy) y verificar que la app siga cargando la bóveda y las voces locales.
2. **Tests del frontend**: montar `vitest` + `@testing-library/react` y escribir el primer test de una función pura (el layout, la exportación a Obsidian, la construcción de nodos). Con eso, la migración de React Flow deja de ser a ciegas.

Movimiento 2 — Migrar a `@xyflow/react` 12 en un worktree (un día)

Después del Movimiento 1 (necesitás los tests para saber que no rompiste nada). Se hace con la rama de trabajo, los árbitros y PR: el `package.json` es lo fácil; el trabajo está en los nodos y los tipos.

Movimiento 3 — Pulir como programa de verdad (una tarde corta, mucho retorno percibido)

- **Bandeja + atajo global + window-state**: los tres juntos cambian la sensación de app de escritorio.
- **Soltar archivos sobre el lienzo**: 1 hora y ya tenés una función que ningún competidor local tiene.
- **Paleta de comandos** con `cmdk` + `fuse.js`.
- **Efectos de ventana** (Mica) y menú nativo: gratis en el core de Tauri.

Lo que NO haría ahora

- **Base embebida** (redb/SQLite): el JSON todavía te sirve y te deja auditar. Primero medí.
- **CRDT** (yjs): es la base de tu fase móvil, pero es un proyecto en sí. Va cuando esa fase empieza.
- **Automerge**: en alpha.
- **elkjs**: licencia EPL/GPL en un producto que distribuís; dagre es MIT y alcanza.
- **Migrar a la 2.0-rc de tauri-specta** sin verificar compatibilidad con Tauri 2.11.
- **MSIX / Microsoft Store**: cuenta paga y empaquetado, sin usuarios todavía.

7 · Fuentes y método

Todo lo de este informe se sondeó el 19/09/2026 contra: `registry.npmjs.org` (versión, fecha de publicación, licencia, `peerDependencies`) · `crates.io/api/v1` (versión estable, fecha, dependencias declaradas y autoría) · `gh api` (existencia, estrellas, actividad, licencia) · y el propio repositorio de NodeFlow (`Cargo.toml`, `package.json`, `tauri.conf.json`, `capabilities/default.json`, y un barrido de `kuzu|tldraw|reactflow|fs:write` sobre el código). Las afirmaciones de la investigación delegada se conservaron sólo cuando el sondeo las confirmó; el resto está en la §0 con su prueba. Los esfuerzos son estimaciones, no mediciones.

